2ème partie

JavaScript langage objet

Jean-Christophe DUBOIS
jean-christophe.dubois@univ-rennes.fr

Objet

- Définition

  Un objet est une instance d'une classe, pouvant également être défini directement par un objet littéral.
  Élément, éventuellement complexe, défini par :

  - un constructeur : permet de créer un nouvel objet et d'initialiser ses caractéristiques
  - des propriétés : variables rattachées à l'objet
  - des méthodes : fonctions définies dans l'objet et permettant de manipuler ses propriétés

  Un objet JavaScript peut modéliser :

  - concept abstrait : date, commande, prix
  - objet concret : produit, client, voiture

Création d'un Objet

- Avec le constructeur Object

```
let monObjet = new Object() ;
monObjet.prop1 = "texte";        // propriété textuelle
monObjet.prop2 = 10;             // propriété numérique
monObjet.prop3 = true;           // propriété booléenne
```

- Objet littéral (le plus courant)

```
let monObjet = {
    prop1: "texte",              // propriété textuelle
    prop2: 10,                   // propriété numérique
    prop3: true                  // propriété booléenne
}
console.log(typeof monObjet);    // "object"
```

Rq: En réalité, monObjet est une variable qui référence un objet

Modification d’un Objet

- Création d'un objet vide

```
let monObjet = { };
```

- Ajout/Modification de propriétés

```
monObjet["prop4"] = 1789;        // méthode par crochet
monObjet.prop5 = "valeur1";      // méth.  accesseur / point
```

La notation pointée est un raccourci de la notation par crochet

- Suppression d'une propriété

```
delete monObjet.prop5;
```

- Accès à une propriété

```
let val1 = monObjet.prop4;       // val1 vaut 1789
```

Propriétés d'un Objet

- Valeurs des propriétés

```
let monObjet = {
    prop1: "texte",                 // propriété textuelle
    prop2: 10                       // propriété numérique
}
Object.values(monObjet) ;           // ["texte", 10]
Object.values(monObjet)[0];         // "texte"
```

- Intitulés des propriétés

```
Object.keys(monObjet);              // ["prop1", "prop2"]
Object.keys(monObjet)[0];           // "prop1"
```

Propriétés d'un Objet

- Paires clé/valeur

```
let monObjet = {
    prop1: "texte",                 // propriété textuelle
    prop2: 10                       // propriété numérique
}
Object.entries(monObjet) ;
            // [["prop1", "texte"], ["prop2", 10]]
Object.entries(monObjet)[0];
            // ["prop1", "texte"]
```

- Vérification de l'existence d'une propriété

```
monObjet.hasOwnProperty("prop1");        // true
```

Propriétés d'un Objet

- Propriétés dynamiques
  - Soit une variable

```
const nomVal= "reponse";
```

  - Le contenu de la variable nomVal peut être utilisé comme intitulé d'une propriété de l'objet

```
let monObjet = {
    prop1: "texte",                 // propriété textuelle
    prop2: 10,                      // propriété numérique
    [nomVal]: true                  // propriété booléenne
}
console.log(monObjet.reponse) ;                 // true
```

Imbrication d'Objets

- Un objet peut avoir pour propriété une valeur de type objet

```
let monObjet = {
    prop1:  {
                unePropA : "valA",
                unePropB : "valB"
            },
    prop2: 10,
    prop3: true
}
console.log(typeof monObjet.prop1);        // "object"
console.log(monObjet.prop1.unePropB);      // "valB"
```

Méthodes d'un Objet

- Fonctions associées aux propriétés de l'objet

```
let monObjet = {
    prop1: 10,
    affiche: function() {
        alert("Je suis un Objet !") ;
    }
}
monObjet.affiche();
```

Opérateur this

- this référence à l'objet appelant.
  Il permet à une méthode d'accéder à ses propriétés

```
let monObjet = {
    prop1: 10,
    affiche: function() {
            alert("Je ne suis pas le n°  " + this.prop1
    + " je suis un Objet libre !") ;
    }
}
monObjet.affiche();
```

Ajout de propriétés/méthodes

- Ajout dynamique de propriétés/méthodes

```
let monObjet = {
    prop1: 10,
    affiche: function() {
            alert("Je ne suis pas le n°  " + this.prop1
    + " je suis un Objet libre !") ;
    }
}
monObjet.prop2 = "nouvelleProp" ;        // propriété ajoutée
monObjet.presente = function() {
    alert(" N°  " + this.prop1 + " avec une " + this.prop2);
    }                                    // méthode ajoutée
monObjet.presente();          // N°  10 avec une nouvelleProp
```

Class et constructor *(syntaxe ES6)*

- Classe avec constructeur, propriétés et méthodes

```
class Planete {                                    // nouvelle déclaration
    constructor (n, r) {
            this.nom = n;
            this.rayon = r;
    }
    toString() { return `${this.nom}
                        a un rayon de ${this.rayon}km`  }
}
let mars = new Planete("Mars", 3389);  // nouvel obj. planète
console.log(mars.toString());    // Mars a un rayon de 3389km
```

- Les méthodes ne sont pas dupliquées dans chaque instance, elles sont stockées dans le prototype

Getters/Accesseurs

- Get : Accès au contenu d'une propriété

```
class Planete {
    constructor (n, r) {
            this._nom = n;          // nom est renommé _nom
            this._rayon = r;        // rayon est renommé _rayon
    }                               // pour éviter la confusion !
    get nom() { return this._nom }      // définition du getteur
}
let mars = new Planete("Mars", 3389);
console.log(mars.nom);              // appel du getteur sans ( )
                                    // Mars
```

Une méthode ne peut pas porter le même nom qu'une propriété

Getters/Accesseurs

- Accès au contenu d'une "propriété" calculée

```
class Planete {
    constructor (n, r) {
        this._nom = n;
        this._rayon = r;
    }
    get perimetre() {                    // calcul de la propriété
        return 2 * Math.PI * this._rayon }
}
let mars = new Planete("Mars", 3389);
console.log(mars.perimetre + "km");      // appel du getteur
                                         //  21293.715km
```

Setters/Mutateurs

- Set : modification du contenu d'une propriété

```
class Planete {
    constructor (n, r) {
        this._nom = n;
        this._rayon = r;                    // _rayon ≠ rayon !
    }
    set rayon(n) { this._rayon = n }        // définition du setteur
}
let mars = new Planete("Mars", 3389);
mars.rayon = 3390;                          // modification de _rayon
```

Une méthode ne peut pas avoir le même nom qu'une propriété

Propriétés statiques

- Static : définition d'une propriété de classe

```
class Planete {
    static systeme = "solaire"; // propriété commune à toutes
                                // les instances de Planete
    constructor (n, r) {
        this._nom = n;
        this._rayon = r;
    }
}
let mars = new Planete("Mars", 3389);
console.log(Planete.systeme);        // Solaire
```

Les propriétés statiques sont appelées exclusivement par une classe

Méthode statique

- Static : définition d'une méthode de classe

```
class Planete {
    constructor (n, r) {
            this.nom = n;
            this.rayon = r;
    }
    static plusGrande( p1, p2) {      // comparaison de 2 planètes
            return (p1.rayon > p2.rayon ? p1 : p2)  }
}
let mars = new Planete("Mars", 3389);
let venus = new Planete("Venus", 6051);
console.log(Planete.plusGrande(mars,venus).nom);       // Venus
```

Les méthodes statiques sont appelées exclusivement par une classe

Héritage : extends et super

- Héritage et appel au super constructeur

```
class PlaneteGaz extends Planete  {
    constructor (n, r, g) {
            super (n, r);          // appel au constructeur de Planete
            this.gaz = g;
    }
}
let jupiter = new PlaneteGaz("Jupiter", 69911, "H");
console.log(jupiter.toString());                // toString() de Planete
                                        // Jupiter a un rayon de 69911km
```

PlaneteGaz est une sous-classe de la super classe Planete

Héritage : extends et super

- Appel d'une méthode parente

```
class PlaneteGaz extends Planete  {
    constructor (n, r, g) {
            super (n, r);
            this.gaz = g;
    }
    toString() {  return `${super.toString()}   // toString() de Planete
                        et est composée de ${this.gaz }` }
}
let jupiter = new PlaneteGaz("Jupiter", 69911, "H");
console.log(jupiter.toString());          // toString() de PlaneteGaz
                        // Jupiter a un rayon de 69911km
                        // et est composée de H
```

Déstructuration d'objet

- Transformation des propriétés d'un objet en variables

```
let maPlanete = {
    nom: "Venus",
    rayon: 6051
}
let nom = maPlanete.nom;            // syntaxe
let rayon = maPlanete.rayon;        // fastidieuse
```

En entourant les propriétés de l'objet avec des accolades { }, les variables seront créées et les valeurs affectées.

```
let { nom, rayon } = maPlanete;     // écriture raccourcie
console.log(nom);       // Venus
console.log(rayon);     // 6051
```

L'objet original maPlanete n'est pas affecté

Déstructuration d'objet

- Déstructuration imbriquée des propriétés d'un objet en variables séparées

```
let taPlanete = {
    nom: "Jupiter",
    rayon: 69911,
    lunes: { europe: "Europe",    // objet lunes imbriqué
             io: "Io" }
};
let { nom, rayon, lunes: { europe, io } } = taPlanete;
    // double déstructuration
    // europe et io récupérées depuis l'objet imbriqué lunes

console.log(nom, rayon, europe, io);
    // Jupiter    69911    Europe    Io
```

Déstructuration d'objet

- Déstructuration avec valeur par défaut et omission

```
let taPlanete = {
    nom: "Jupiter",
    lunes: { europe: "Europe",    // objet lunes imbriqué
             io: "Io" }
};
let { nom, rayon=0, lunes: { io } } = taPlanete;
    // nom est conservé
    // rayon, absent dans l'objet → valeur par défaut
    // Europe est ignoré

console.log(nom, rayon, io);    // Jupiter 0 Io
```

Déstructuration d'objet

- Renommage de variable grâce à l'usage de :

```
let maPlanete = {
    nom: "Venus",
    rayon: 6051
}
let { nom: nomP, rayon } = maPlanete;
                        // nomP sera utilisé à la place de nom
alert(nomP);            // Venus
```

Fonctions fléchées et this

- Les fonctions fléchées () => {} n'ont **pas leur propre this**, elles héritent du this de leur contexte parent.

```
class Planete {
    constructor (n, r) {
        this.nom = n;
        this.rayon = r;
    }
    affiche = ()=> console.log(`${this.nom} - ${this.rayon}km`);
  }
}
let mars = new Planete("Mars", 3389);
mars.affiche();                    // undefined
```

- this ne pointe pas sur mars, mais sur le contexte global, window

Fonctions fléchées et this

- Les fonctions fléchées () => {} n'ont **pas leur propre this**, elles héritent du this de leur contexte parent.

```
class Planete {
    constructor (n, r) {    this.nom = n; this.rayon = r;   }
    affiche() {
            setTimeout(() => {
                    console.log(`${this.nom} - ${this.rayon}km`);
            }, 100);
    }
}
let mars = new Planete("Mars", 3389);
mars.affiche();                    // Mars - 3389 après 100ms
```

- la fonction fléchée hérite du this de la méthode, i.e. mars

Opérateur in

- L'opérateur in teste l'existence d'une propriété :
  - retourne true si la propriété existe
  - false sinon

```
alert( "rayon" in maPlanete);          // true
alert( "couleur" in maPlanete);        // false
```

- Parcours des propriétés avec une boucle for … in

```
for (let valeur in maPlanete)
    alert(maPlanete[valeur]);
//  Affichages successifs de Venus et de 6051
```

Imbrication d'Objets

- Accès aux propriétés d'un objet et à leurs propres propriétés

```
let monObjet = {
    prop1:  {       propA : "valA",
                    propB : "valB" },
    prop2:  {       propC : "valC",
                    propD : "valD" }
}
for (let prop in monObjet) {
    console.log (prop);             //   prop1 … prop2
    for (let ssProp in monObjet[prop])
        console.log (ssProp);       //   propA, propB …
                                    //   propC, propD
 }
```

JSON

- JSON (JavaScript Object Notation) est un format de données qui permet de stocker et échanger des informations de manière structurée
- Les données sont organisées sous la forme d'une paire : "clé" : valeur
- Les valeurs des fichiers JSON (.json) peuvent contenir des chaînes de caractères "…", des nombres, des booléens, des objets {…}, des tableaux […] ou encore des valeurs null.

```
{
    "nom" : "Jupiter",
    "lunes":  ["Europe", "Ganymède",  "Io"],
    "attributs":  {  "rayon":  69911,
                     "tellurique": false,
                     "gravité": null }
}
```

JSON → Objet JS

- parse() : conversion d'une chaîne JSON (délimitée par des ') en objet JS

```
const txt = '{        "nom" : "Jupiter",
                      "lunes":  ["Europe", "Ganymède",  "Io"],
                      "attributs":  {  "rayon":  69911,
                                       "tellurique": false,
                                       "gravité": null }
 } ';
const planete = JSON.parse(txt);
console.log(planete.nom);                          // Jupiter
console.log(planete.lunes[2]);                     // Io
console.log(planete.attributs.rayon);              // 69911
```

Objet JS → JSON

- stringify() : conversion d'un objet JS en chaîne JSON

```
const objet = {     nom : "Jupiter",
                    lunes:  [ "Europe", "Ganymède",  "Io"],
                    attributs :  {     rayon:  69911,
                                       tellurique: false,
                                       gravité: null }
 };
const planeteJson = JSON.stringify(objet);
console.log(planeteJson);
     // {"nom": "Jupiter", "lunes": ["Europe", "Ganymède", "Io"],
          "attributs": {"rayon": 69911, "tellurique": false, "gravité": null} }
```